

WAZUH

Wazuh est une plateforme open source SIEM/XDR qui centralise la surveillance de l'infrastructure. Il collecte et analyse les logs en temps réel depuis des agents déployés sur chaque serveur, détecte les menaces et les affiche dans un dashboard centralisé.

- [INSTALLATION](#)
- [INTEGRATION LDAP](#)
- [FIM](#)
- [Wazuh — Incident Disque Plein](#)
- [Wazuh — Active Response](#)
- [Wazuh — Intégration GLPI](#)

INSTALLATION

WAZUH SIEM

Documentation technique d'installation

Compte-rendu de session réelle

25 mars 2026

Paramètre	Valeur
Serveur	srvwazuh
FQDN	srvwazuh.c1-04.lan
IP statique	172.18.5.176/24
Passerelle	172.18.5.254
DNS	172.18.5.1 / 1.1.1.1
Interface réseau	enp6s18
OS	Ubuntu Server 22.04 LTS
Version Wazuh	4.14 (dernière stable)
Mode déploiement	All-in-One (Indexer + Server + Dashboard)
RAM disponible	7,8 Go
Environnement	Proxmox VE — VM KVM (q35)

1. Contexte et objectif

Cette documentation retrace l'installation complète de Wazuh SIEM en configuration All-in-One sur une VM Proxmox, réalisée en session interactive le 25 mars 2026. Elle inclut toutes les commandes exécutées, les erreurs rencontrées et leurs résolutions, dans l'ordre chronologique exact de la session.

L'objectif est de disposer d'un SIEM opérationnel capable de surveiller l'infrastructure existante : serveur Active Directory, hôtes Docker (Vaultwarden, Nextcloud) et autres serveurs du domaine c1-04.lan.

1.1 Configuration VM Proxmox

Onglet	Paramètre	Valeur choisie
System	Machine	q35
System	BIOS	SeaBIOS
System	SCSI Controller	VirtIO SCSI single
System	Qemu Agent	Activé
CPU	Cores	4
CPU	Type	host
Memory	RAM	8192 Mo
Memory	Ballooning	Désactivé
Network	Bridge	vmbr0
Network	Model	VirtIO
Disk	Taille	50 Go
Disk	Bus	VirtIO Block

⚠ Le Ballooning device a été désactivé volontairement. OpenSearch (JVM) alloue un heap fixe au démarrage — le ballooning peut provoquer des OOM kills imprévisibles.

2. Préparation du système

2.1 Vérification de la configuration réseau

Première commande exécutée pour vérifier l'état réseau de la VM après installation d'Ubuntu :

```
ip a
```

```
2: enp6s18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
```

```
inet 172.18.5.176/24 brd 172.18.5.255 scope global enp6s18
```

```
valid_lft forever preferred_lft forever
```

i La mention 'valid_lft forever' indique que l'IP est bien statique malgré l'apparence DHCP. L'IP avait été saisie manuellement pendant l'installateur Ubuntu.

Vérification du fichier netplan pour confirmer la configuration statique :

```
cat /etc/netplan/50-cloud-init.yaml
```

```
network:
```

```
version: 2
```

```
ethernets:
```

```
enp6s18:
```

```
addresses:
```

```
- "172.18.5.176/24"
```

```
nameservers:
```

```
addresses:
```

```
- 172.18.5.1
```

```
- 1.1.1.1
```

```
routes:
```

```
- to: "default"
```

```
via: "172.18.5.254"
```

□ IP statique confirmée. Aucune modification nécessaire.

⚠ Le fichier s'appelait 50-cloud-init.yaml et non 00-installer-config.yaml comme attendu. Ubuntu 22.04 avec cloud-init génère ce nom de fichier.

2.2 Mise à jour du système

```
sudo apt update && sudo apt upgrade -y
```

□ Système mis à jour sans erreur.

2.3 Configuration du FQDN

Vérification du hostname avant configuration :

```
hostname && hostname -f  
srvwazuh  
srvwazuh
```

Le hostname -f retournait uniquement 'srvwazuh' sans domaine. Wazuh liant ses certificats au FQDN, il était nécessaire de configurer le nom de domaine complet avant l'installation. Le domaine interne de l'infrastructure est c1-04.lan.

```
sudo nano /etc/hosts
```

Modification de la ligne :

```
127.0.1.1 srvwazuh
```

Remplacée par :

```
127.0.1.1 srvwazuh.c1-04.lan srvwazuh
```

```
hostname -f
```

□ FQDN correctement configuré.

2.4 Désactivation du swap

OpenSearch exige que le swap soit désactivé. Avec le swap actif, la JVM peut se retrouver à utiliser de la mémoire swap sans le savoir, provoquant des latences catastrophiques et des timeouts entre composants Wazuh.

```
sudo swapoff -a
sudo sed -i '/swap/d' /etc/fstab
free -h
```

```
total used free shared buff/cache available
Mem: 7,8Gi 466Mi 6,8Gi 1,2Mi 734Mi 7,3Gi
Swap: 0B 0B 0B
```

□ Swap désactivé et supprimé de fstab (persistant après redémarrage).

2.5 Paramètre kernel vm.max_map_count

OpenSearch nécessite un minimum de 262144 zones mémoire mappables. La valeur par défaut du kernel Linux (65530) est insuffisante — OpenSearch refuse de démarrer si cette valeur n'est pas correcte.

```
echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
vm.max_map_count = 262144
```

□ Paramètre appliqué immédiatement et persistant.

2.6 Vérification des outils requis

```
curl --version && wget --version | head -1  
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13...  
GNU Wget 1.21.4 built on linux-gnu.
```

□ curl et wget disponibles. Prérequis validés.

3. Installation Wazuh

3.1 Téléchargement du script et de la configuration

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
curl -sO https://packages.wazuh.com/4.14/config.yml
ls -lh wazuh-install.sh config.yml
```

```
-rw-rw-r-- 1 user_wazuh user_wazuh 636 mars 25 10:11 config.yml
-rw-rw-r-- 1 user_wazuh user_wazuh 195K mars 25 10:11 wazuh-install.sh
```

□ Fichiers téléchargés correctement.

3.2 Configuration du fichier config.yml

Le fichier config.yml contient des placeholders à remplacer par l'IP réelle. Pour un déploiement all-in-one, les 3 sections (indexer, server, dashboard) pointent vers la même IP.

```
nano config.yml
```

Contenu final après modification :

```
nodes:
indexer:
- name: node-1
ip: "172.18.5.176"

server:
```

```
- name: wazuh-1  
ip: "172.18.5.176"
```

```
dashboard:  
- name: dashboard  
ip: "172.18.5.176"
```

□ Les 3 IPs configurées à 172.18.5.176.

3.3 Génération des certificats TLS

```
bash wazuh-install.sh --generate-config-files
```

```
ls -lh wazuh-install-files.tar  
-rw----- 1 root root 11K mars 25 10:13 wazuh-install-files.tar
```

□ Certificats générés. Le fichier wazuh-install-files.tar contient tous les certificats TLS — à conserver précieusement.

3.4 Installation du Wazuh Indexer

```
bash wazuh-install.sh --wazuh-indexer node-1
```

□ Installation terminée : 'installation finished'

3.5 Démarrage du cluster

```
bash wazuh-install.sh --start-cluster
```

❑ Cluster démarré : 'cluster started'

3.6 Vérification de l'indexer

Récupération du mot de passe admin :

```
tar -axf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt -O | grep -P '"admin"' -A 1
```

Test de connexion à l'indexer :

```
curl -k -u admin:MOT_DE_PASSE https://127.0.0.1:9200
```

❑ Erreur #1 — Connexion refusée sur 127.0.0.1:9200

Commande exécutée :

```
curl -k -u admin:MOT_DE_PASSE https://127.0.0.1:9200
```

Erreur obtenue :

```
curl: (7) Failed to connect to 127.0.0.1 port 9200 after 0 ms: Couldn't connect to server
```

Cause :

OpenSearch écoutait sur l'IP de l'interface (172.18.5.176) et non sur 127.0.0.1. Dans config.yml, l'IP réelle avait été renseignée, donc OpenSearch bind sur cette IP spécifiquement.

Solution :

```
curl -k -u admin:MOT_DE_PASSE https://172.18.5.176:9200
```

Réponse correcte après correction :

```
{
  "name" : "node-1",
  "cluster_name" : "wazuh-indexer-cluster",
  "cluster_uuid" : "Pl8Z2_TiSSmTZ3U2PR4k0w",
  "version" : { "number" : "7.10.2", ... },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
```

❑ Indexer opérationnel.

3.7 Installation du Wazuh Server

```
bash wazuh-install.sh --wazuh-server wazuh-1
```

□ Installation terminée : 'installation finished'

3.8 Installation du Wazuh Dashboard

```
bash wazuh-install.sh --wazuh-dashboard dashboard
```

□ Installation terminée. Mots de passe affichés en fin de script et notés.

⚠ Les mots de passe ne s'affichent qu'une seule fois à la fin du script. Les noter immédiatement dans un gestionnaire de mots de passe.

4. Vérification finale

4.1 État des services

```
for svc in wazuh-indexer wazuh-manager filebeat wazuh-dashboard; do echo -n "$svc : "; systemctl is-active $svc; done
```

```
wazuh-indexer : active
```

```
wazuh-manager : active
```

```
filebeat : active
```

```
wazuh-dashboard : active
```

□ Les 4 services sont actifs et démarrés automatiquement au boot.

4.2 Accès au Dashboard

Paramètre	Valeur
URL	https://172.18.5.176
Utilisateur	admin
Mot de passe	(généré lors de l'installation — noter dans gestionnaire de MDP)
Certificat	Auto-signé — avertissement SSL normal en environnement interne

5. Récapitulatif des erreurs rencontrées

#	Erreur	Cause	Résolution
1	curl: (7) Failed to connect to 127.0.0.1:9200	OpenSearch bind sur l'IP réelle (172.18.5.176) et non sur localhost car config.yml spécifiait l'IP réelle	Utiliser https://172.18.5.176:9200 au lieu de 127.0.0.1

i L'erreur rencontré était d'erreur de configuration/frappe et non d'erreur d'installation. L'installation elle-même s'est déroulée sans problème.

INTEGRATION LDAP

Wazuh SIEM

Intégration Active Directory via LDAP

Documentation d'installation et de configuration

Version Wazuh	4.14.4 (stable)
Domaine AD	c1-04.lan
Serveur Wazuh	172.18.5.176
DC / LDAP	172.18.5.1:389
Compte de service	CN=LdapService,CN=Users,DC=c1-04,DC=lan
Date	25/03/2026

1. Contexte et objectif

L'objectif est de permettre aux utilisateurs Active Directory du domaine c1-04.lan de s'authentifier directement sur le dashboard Wazuh avec leurs identifiants de domaine, sans créer de comptes locaux supplémentaires.

L'architecture repose sur trois couches :

- OpenSearch Security (wazuh-indexer) : gère l'authentification LDAP et les rôles OpenSearch
- Wazuh API (port 55000) : gère les permissions d'accès aux agents et aux données Wazuh
- Active Directory : source d'identité et de groupes

2. Prérequis AD

2.1 Compte de service LDAP

Créer un compte de service dédié dans l'AD avec des droits en lecture sur l'annuaire. Ce compte est utilisé par Wazuh pour interroger le LDAP.

Compte	CN=LdapService,CN=Users,DC=c1-04,DC=lan
Rôle	Lecture seule sur l'annuaire

2.2 Groupes AD

Deux groupes de type Sécurité / Globale ont été créés dans l'OU Groupes :

WazuhAdmins	CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan
WazuhUsers	CN=WazuhUsers,OU=Groupes,DC=c1-04,DC=lan
Type	Groupe de sécurité — Étendue Globale

Les utilisateurs autorisés à accéder à Wazuh sont ajoutés dans l'un de ces deux groupes selon leur niveau d'accès souhaité.

3. Configuration OpenSearch Security

OpenSearch Security est le plugin de sécurité natif d'OpenSearch, intégré par défaut dans le déploiement Wazuh. Il assure trois fonctions essentielles : le chiffrement TLS de toutes les communications inter-composants (Indexer, Server, Dashboard), l'authentification des utilisateurs via un backend interne ou externe (dans notre cas, l'Active Directory du domaine c1-04.lan via LDAP), et l'autorisation par un système de rôles (RBAC) permettant de restreindre les actions selon le profil de l'utilisateur. Les groupes AD WazuhAdmins et WazuhUsers sont ainsi mappés sur les rôles Wazuh correspondants, garantissant qu'un utilisateur standard puisse consulter les dashboards sans pouvoir modifier la configuration. Les certificats TLS générés lors de l'installation sont utilisés par ce plugin pour établir un chiffrement mutuel (mTLS) entre chaque composant, assurant l'intégrité et la confidentialité des données de sécurité transitant dans la stack.

3.1 Fichier config.yml

Fichier : /etc/wazuh-indexer/opensearch-security/config.yml

Modifications apportées dans la section authc — bloc ldap :

```

ldap:
description: "Authenticate via LDAP or Active Directory"
http_enabled: true
transport_enabled: true
order: 5
http_authenticator:
type: basic
challenge: false
authentication_backend:
type: ldap
config:
enable_ssl: false
enable_start_tls: false
enable_ssl_client_auth: false
verify_hostnames: false
hosts:
- 172.18.5.1:389
bind_dn: 'CN=LdapService,CN=Users,DC=c1-04,DC=lan'
password: '<mot_de_passe_ldapservice>'
userbase: 'DC=c1-04,DC=lan'
usersearch: '(&(sAMAccountName={0}))((memberOf=CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan)(memberOf=CN=WazuhUsers,OU=Groupes,DC=c1-04,DC=lan)))'
username_attribute: sAMAccountName

```

Le usersearch filtre l'accès : seuls les membres de WazuhAdmins ou WazuhUsers peuvent s'authentifier. Les autres comptes AD sont rejetés même avec le bon mot de passe.

Modifications apportées dans la section authz — bloc roles_from_myldap :

```

roles_from_myldap:
description: "Authorize via LDAP or Active Directory"
http_enabled: true
transport_enabled: true
authorization_backend:
type: ldap
config:
enable_ssl: false
verify_hostnames: false
hosts:
- 172.18.5.1:389
bind_dn: 'CN=LdapService,CN=Users,DC=c1-04,DC=lan'
password: '<mot_de_passe_ldapservice>'
rolebase: 'OU=Groupes,DC=c1-04,DC=lan'
rolesearch: '(member={0})'
rolename: dn
resolve_nested_roles: true
userbase: 'DC=c1-04,DC=lan'
usersearch: '(sAMAccountName={0})'

```

3.2 Fichier roles_mapping.yml

Fichier : /etc/wazuh-indexer/opensearch-security/roles_mapping.yml

Ajout des groupes AD dans les rôles OpenSearch existants :

```
all_access:
backend_roles:
- "admin"
- "CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan"
```

```
readall:
backend_roles:
- "readall"
- "CN=WazuhUsers,OU=Groupes,DC=c1-04,DC=lan"
```

```
kibana_user:
backend_roles:
- "kibanauser"
- "CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan"
- "CN=WazuhUsers,OU=Groupes,DC=c1-04,DC=lan"
```

```
wazuh_ui_admin:
backend_roles:
- "CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan"
```

```
wazuh_ui_user:
backend_roles:
- "CN=WazuhUsers,OU=Groupes,DC=c1-04,DC=lan"
```

3.3 Application de la configuration

La configuration OpenSearch Security ne s'applique pas via un simple redémarrage. Il faut utiliser le script securityadmin.sh :

Problème rencontré : le wazuh-indexer n'écoute pas sur 127.0.0.1 mais sur l'IP du serveur (172.18.5.176). L'option -h doit pointer vers cette IP, pas localhost.

```
export JAVA_HOME=/usr/share/wazuh-indexer/jdk
sudo -E bash /usr/share/wazuh-indexer/plugins/opensearch-security/tools/securityadmin.sh \
-cd /etc/wazuh-indexer/opensearch-security/ \
```

```
-icl -nhnv \  
-cacert /etc/wazuh-indexer/certs/root-ca.pem \  
-cert /etc/wazuh-indexer/certs/admin.pem \  
-key /etc/wazuh-indexer/certs/admin-key.pem \  
-h 172.18.5.176
```

Important : utiliser `sudo -E` pour conserver la variable `JAVA_HOME` lors de l'élévation de privilèges. Sans le `-E`, le script échoue avec un avertissement `JAVA_HOME not set`.

4. Configuration Wazuh API (port 55000)

4.1 Problème identifié

Après configuration d'OpenSearch, l'authentification AD fonctionnait sur le dashboard mais les agents n'étaient pas visibles. Le message d'erreur était :

Require administrator privileges: User has no administrator role in the selected API connection.

Le fichier `/usr/share/wazuh-dashboard/data/wazuh/config/wazuh.yml` contient `run_as: true`, ce qui active le mode d'emprunt d'identité : l'API Wazuh tente d'exécuter les requêtes sous l'identité de l'utilisateur connecté. Or, l'API Wazuh a sa propre gestion des utilisateurs, séparée d'OpenSearch.

4.2 Solution : Security Rules Wazuh API

La solution est de créer une Security Rule dans l'API Wazuh qui matche les utilisateurs AD via leur `backend_role` (le groupe AD transmis par OpenSearch) et leur attribue le rôle `administrator` (id: 1) de l'API Wazuh.

Étape 1 — Obtenir un token API :

```
TOKEN=$(curl -k -u wazuh-wui:"<mot_de_passe_wui>" \  
-X POST "https://172.18.5.176:55000/security/user/authenticate?raw=true")
```

Étape 2 — Créer la rule :

```
curl -k -X POST "https://172.18.5.176:55000/security/rules" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "wui_ad_wazuh_admins",  
  "rule": {  
    "FIND": {  
      "backend_roles": "CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan"  
    }  
  }  
}
```

```
}'
```

Étape 3 — Lier la rule au rôle administrator (id: 1). La rule créée retourne un id (ex: 100) :

```
curl -k -X POST "https://172.18.5.176:55000/security/roles/1/rules?rule_ids=100" \
-H "Authorization: Bearer $TOKEN"
```

La rule basée sur backend_roles matche tous les membres du groupe WazuhAdmins, quel que soit leur sAMAccountName. Il n'est pas nécessaire de créer une rule par utilisateur.

5. Erreurs rencontrées et solutions

Erreur	Solution
WARNING: JAVA_HOME is not set	Utiliser export JAVA_HOME=/usr/share/wazuh-indexer/jdk puis sudo -E pour conserver la variable
Seems there is no OpenSearch running on 127.0.0.1:9200	Le wazuh-indexer écoute sur l'IP du serveur (172.18.5.176), pas sur localhost. Utiliser -h 172.18.5.176
Agents non visibles après connexion AD	Rôles OpenSearch OK mais droits API Wazuh manquants. Ajouter les rôles wazuh_ui_admin et wazuh_ui_user dans roles_mapping.yml
User has no administrator role in the selected API connection	Créer une Security Rule dans l'API Wazuh matchant le backend_role du groupe AD et la lier au rôle administrator (id: 1)
404 Not Found sur /security/roles_mappings	L'endpoint correct dans cette version est /security/rules — la gestion des mappings se fait via les rules, pas les roles_mappings

6. Vérification et tests

6.1 Test de connectivité LDAP

Avant toute configuration, vérifier que le serveur Wazuh peut interroger le DC :

```
ldapsearch -x -H ldap://172.18.5.1 \
-D "CN=LdapService,CN=Users,DC=c1-04,DC=lan" \
-w "<mot_de_passe>" \
-b "DC=c1-04,DC=lan" \
"(sAMAccountName=nom_utilisateur_test)"
```

Un retour avec les attributs de l'utilisateur (dn, memberOf, sAMAccountName...) confirme que la connexion LDAP fonctionne.

6.2 Vérification des rôles dans le dashboard

Après connexion avec un compte AD, cliquer sur l'avatar en haut à droite du dashboard pour vérifier les rôles assignés. Les rôles attendus pour un admin sont :

- wazuh_ui_admin
- all_access
- kibana_user
- own_index

Les Backend roles doivent afficher les DNs des groupes AD correspondants (CN=WazuhAdmins,OU=Groupes,DC=c1-04,DC=lan...).

7. Notes de sécurité

Le mot de passe du compte LdapService est stocké en clair dans config.yml. Restreindre les permissions du fichier et envisager une rotation régulière du mot de passe.

Recommandations complémentaires :

- Passer en LDAPS (port 636) en production pour chiffrer les échanges d'authentification
- Le compte LdapService ne doit avoir que des droits en lecture sur l'annuaire
- Conserver une sauvegarde du répertoire opensearch-security avant toute modification
- Tester la configuration avec un compte non-membre des groupes Wazuh pour vérifier que l'accès est bien refusé

8. Commandes utiles

Version Wazuh	/var/ossec/bin/wazuh-control info
Packages installés	apt list --installed grep wazuh
Port d'écoute indexer	sudo ss -tlnp grep java

Logs indexer	<code>sudo tail -f /var/log/wazuh-indexer/wazuh-indexer-cluster.log</code>
Appliquer config security	<code>sudo -E bash /usr/share/wazuh-indexer/plugins/opensearch-security/tools/securityadmin.sh ...</code>
Token API Wazuh	<code>curl -k -u wazuh-wui:<mdp> -X POST https://172.18.5.176:55000/security/user/authenticate?raw=true</code>
Lister les rules API	<code>curl -k -X GET https://172.18.5.176:55000/security/rules -H "Authorization: Bearer \$TOKEN"</code>

FIM

FILE INTEGRITY MONITORING (FIM) REALTIME

1. Contexte et objectif

Le File Integrity Monitoring (FIM) de Wazuh surveille les modifications apportées aux fichiers sensibles du système (permissions, contenu, propriétaire). Par défaut, la configuration surveille /etc, /usr/bin, /sbin et /boot en mode scan périodique toutes les 12 heures.

L'objectif est de passer en mode realtime pour détecter instantanément toute modification (quelques secondes), en utilisant inotify sous Linux, et de déployer cette configuration de manière centralisée sur tous les agents Debian 13 via le système de groupes Wazuh.

2. Diagnostic — Problème initial

Après modification de /etc/passwd sur un agent, l'alerte n'apparaissait pas dans le dashboard Wazuh. Examen de la configuration syscheck sur le manager :

```
<!-- ossec.conf manager — configuration initiale -->
```

```
<syscheck>
```

```
<disabled>no</disabled>
```

```
<frequency>43200</frequency> <!-- scan toutes les 12h -->
```

```
<scan_on_start>yes</scan_on_start>
```

```
<!-- Pas de realtime="yes" — mode scan périodique uniquement -->
```

```
<directories>/etc,/usr/bin,/usr/sbin</directories>
```

```
<directories>/bin,/sbin,/boot</directories>
```

```
</syscheck>
```

⚠ Sans `realtime="yes"`, Wazuh effectue un scan périodique toutes les 12h. Les modifications ne sont détectées qu'au prochain scan programmé, pas instantanément.

3. Solution — Déploiement centralisé via Agent Groups

Plutôt que de modifier la configuration sur chaque agent individuellement, Wazuh permet de pousser une configuration commune à un groupe d'agents via le fichier `agent.conf`. Tous les agents Debian 13 appartiennent au groupe `Debian13`.

3.1 Identification du groupe des agents

```
sudo /var/ossec/bin/agent_groups -i 002 -s
```

```
The agent 'Agent-Zabbix' with ID '002' belongs to groups: Debian13.
```

! Tous les agents Debian 13 appartiennent au groupe `Debian13` — c'est ce fichier qui doit être modifié, pas le groupe default.

3.2 Configuration du fichier `agent.conf`

Fichier modifié : `/var/ossec/etc/shared/Debian13/agent.conf`

```
<agent_config os="Debian">
```

```
<syscheck>
```

```
<directories realtime="yes" check_all="yes" report_changes="yes">/etc</directories>
```

```
<directories realtime="yes" check_all="yes">/usr/bin,/usr/sbin,/bin,/sbin</directories>
```

```
</syscheck>
```

```
</agent_config>
```

Attribut	Description
realtime="yes"	Utilise inotify pour la détection instantanée des modifications (quelques secondes)
check_all="yes"	Vérifie hash MD5/SHA1/SHA256, permissions, propriétaire, taille, date de modification
report_changes="yes"	Enregistre le diff du contenu modifié (particulièrement utile pour /etc/passwd, /etc/sudoers)
os="Debian"	Filtre l'application aux agents Debian uniquement — les agents Windows ignorent ce bloc automatiquement

3.3 Validation de la configuration

```
sudo /var/ossec/bin/verify-agent-conf
```

```
verify-agent-conf: Verifying [etc/shared/default/agent.conf]
```

```
verify-agent-conf: OK
```

```
verify-agent-conf: Verifying [etc/shared/Debian13/agent.conf]
```

```
verify-agent-conf: OK
```

□ XML valide sur les deux groupes. La configuration est poussée automatiquement aux agents dans les minutes suivantes, sans redémarrage nécessaire.

4. Test et vérification

4.1 Déclenchement d'une alerte FIM

```
# Sur un agent Debian 13 — déclencher une modification
```

```
sudo nano /etc/passwd
```

Modifier le fichier (Rajouter quelque chose a la fin du fichier)

```
# Sur le manager — vérifier en temps réel
```

```
sudo tail -f /var/ossec/logs/alerts/alerts.json | grep passwd
```

□ Alerte FIM générée en quelques secondes après la modification.

4.2 Vérification dans le dashboard

Les alertes FIM remontent automatiquement dans le dashboard Wazuh :

- Threat Intelligence → File Integrity Monitoring : événements FIM avec fichier modifié, type de changement (modified/added/deleted), hash avant/après
- Security Events : filtrer par rule.groups: syscheck pour voir les alertes FIM en temps réel

□ Alerte confirmée dans le dashboard — FIM realtime opérationnel sur tous les agents Debian 13.

5. Architecture de déploiement FIM

Paramètre	Valeur
Mécanisme realtime	inotify (kernel Linux)
Délai de détection	2 à 5 secondes après modification
Mode de déploiement	Centralisé via agent.conf — groupe Debian13
Répertoires surveillés	/etc (realtime + diff), /usr/bin, /usr/sbin, /bin, /sbin
Agents concernés	Tous les agents du groupe Debian13 (Debian 13)
Agents exclus	Agents Windows — filtre os="Debian"
Vérification	verify-agent-conf → OK

Wazuh — Incident Disque Plein

1. Contexte et symptômes

Au démarrage du serveur, les services Wazuh ne démarraient plus. Le service wazuh-indexer échouait avec une erreur Java indiquant que l'espace disque était épuisé, le service wazuh-manager tombait en timeout, et le wazuh-dashboard s'arrêtait faute d'indexer disponible.

Erreur observée dans les journaux système :

```
Exception in thread "main" java.nio.file.FileSystemException:
/tmp/opensearch-17001579755747548849: No space left on device
```

2. Diagnostic

2.1 Vérification de l'espace disque

bash

```
df -h
```

Résultat :

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv	31G	29G	0	100%	/

Le disque root était plein à 100%.

2.2 Identification de la cause

bash

```
sudo du -sh /var/* 2>/dev/null | sort -rh | head -10  
sudo du -sh /var/ossec/queue/vd/feed/
```

Résultat :

```
14G   /var/ossec  
7,9G  /var/ossec/queue/vd/feed/
```

Le dossier `/var/ossec/queue/vd/feed/` contenait les fichiers RocksDB du Vulnerability Detector. Ces fichiers `.sst` s'étaient accumulés sans compaction automatique et représentaient 7,9 Go.

3. Résolution

3.1 Suppression des fichiers RocksDB

Ces fichiers sont reconstruits automatiquement au prochain démarrage de Wazuh. Leur suppression ne présente pas de risque pour la configuration.

bash

```
sudo find /var/ossec/queue/vd/feed/ -name "*.sst" -delete  
sudo find /var/ossec/queue/vd/feed/ -name "LOG.old.*" -delete
```

3.2 Nettoyage complémentaire

bash

```
sudo journalctl --vacuum-size=200M  
sudo apt clean  
sudo find /var/ossec/logs/ -name "*.gz" -mtime +30 -delete
```

3.3 Extension du volume logique

Le volume de 31G était insuffisant pour un déploiement Wazuh all-in-one. Le groupe de volumes disposait de 31G libres non alloués.

Vérification de l'espace disponible :

bash

```
sudo vgdisplay ubuntu-vg
```

VG Size	<62,00 GiB
Alloc PE / Size	7935 / <31,00 GiB
Free PE / Size	7936 / 31,00 GiB

Extension du volume logique puis du système de fichiers :

bash

```
sudo lvextend -L +30G /dev/ubuntu-vg/ubuntu-lv  
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
```

Résultat après extension :

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv	61G	22G	37G	37%	/

3.4 Redémarrage des services

Les services doivent être redémarrés dans l'ordre suivant : wazuh-indexer en premier, puis wazuh-manager, puis wazuh-dashboard.

bash

```
sudo systemctl start wazuh-indexer  
sudo systemctl start wazuh-manager  
sudo systemctl start wazuh-dashboard
```

4. Erreurs rencontrées

Erreur	Cause	Solution
No space left on device	Disque root plein à 100%	Suppression fichiers RocksDB + extension LV
Espace non libéré après suppression	Processus tenant encore les fichiers ouverts	Arrêt complet des services avant suppression
vgdisplay : permission refusée	Exécuté sans sudo	Relancer avec sudo

5. Prévention

La taille minimale recommandée pour un SIEM Wazuh all-in-one est de 50 Go, avec 100 Go conseillés en production. Le dossier `/var/ossec/queue/vd/feed/` peut grossir rapidement lorsque le Vulnerability Detector est actif. Il est recommandé de mettre en place une supervision de l'espace disque avec une alerte à 80% d'occupation.

6. Commandes utiles

bash

```
# Surveiller l'espace disque
df -h /

# Trouver les répertoires volumineux
sudo du -sh /var/* 2>/dev/null | sort -rh | head -10

# Vérifier l'espace disponible sur le groupe de volumes
sudo vgdisplay ubuntu-vg

# Supprimer les anciens logs ossec
sudo find /var/ossec/logs/ -name "*.gz" -mtime +30 -delete

# Réduire les journaux systemd
```

```
sudo journalctl --vacuum-size=200M
```

Wazuh — Active Response

1. Contexte et objectif

L'Active Response Wazuh permet d'exécuter des actions automatiques sur les agents lorsqu'une alerte se déclenche. L'objectif est d'automatiser la réponse aux incidents de sécurité sans intervention humaine : bloquer une IP en cas de brute force SSH, désactiver un compte en cas de tentative sudo non autorisée, ou ajouter une IP dans le fichier hosts.deny.

2. Architecture

Agent (SRV-DOCKER)

↓ logs SSH/sudo via journald

Manager (srvwazuh)

↓ règle détectée (ex: 5763)

↓ commande envoyée à l'agent

Agent

↓ exécute le binaire active-response

↓ ex: iptables -I INPUT -s IP -j DROP

Les binaires disponibles sur les agents se trouvent dans `/var/ossec/active-response/bin/`.

3. Configuration

3.1 Prérequis — Journald sur agents Debian 13

Les agents Debian 13 n'ont pas de fichier `/var/log/auth.log`. Les logs SSH passent par journald. Il faut ajouter la configuration suivante dans `/var/ossec/etc/shared/Debian13/agent.conf` :

xml

```

<agent_config os="Debian">
  <syscheck>
    <directories realtime="yes" check_all="yes" report_changes="yes">/etc</directories>
    <directories realtime="yes" check_all="yes">/usr/bin,/usr/sbin,/bin,/sbin</directories>
  </syscheck>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>
</agent_config>

```

3.2 Configuration ossec.conf

Ajout dans /var/ossec/etc/ossec.conf sur le manager :

xml

```

<!-- Brute force SSH → blocage IP via iptables -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>180</timeout>
</active-response>

<!-- Tentative connexion root SSH → blocage IP -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5720</rules_id>
  <timeout>180</timeout>
</active-response>

<!-- Sudo non autorisé → désactivation compte -->
<active-response>
  <command>disable-account</command>
  <location>local</location>
  <rules_id>5405</rules_id>
  <timeout>180</timeout>

```

```
</active-response>
```

```
<!-- Brute force SSH → ajout dans hosts.deny -->
```

```
<active-response>
```

```
  <command>host-deny</command>
```

```
  <location>local</location>
```

```
  <rules_id>5763</rules_id>
```

```
  <timeout>180</timeout>
```

```
</active-response>
```

4. Règles déclenchantes

Règle ID	Description	Niveau	Action déclenchée
5763	Brute force SSH	10	firewall-drop + host-deny
5720	Tentative connexion root SSH	5	firewall-drop
5405	Sudo non autorisé	5	disable-account
651	Confirmation blocage firewall-drop	3	—

5. Test — Brute force SSH

5.1 Lancement de l'attaque

Depuis une machine Kali :

```
bash
```

```
hydra -l user_docker -P /usr/share/wordlists/rockyou.txt ssh://172.18.5.172 -t 4 -V
```

5.2 Surveillance des alertes sur le manager

```
bash
```

```
sudo tail -f /var/ossec/logs/alerts/alerts.log | grep -E "5763|firewall|brute"
```

5.3 Résultat observé

La règle 5763 se déclenche après 8 tentatives d'authentification échouées. La commande firewall-drop est exécutée sur l'agent et l'IP attaquante est bloquée via iptables.

Vérification du blocage sur l'agent SRV-DOCKER :

bash

```
sudo iptables -L INPUT -n | grep DROP
```

5.4 Logs active-response sur l'agent

bash

```
sudo tail -f /var/ossec/logs/active-responses.log
```

6. Comportement du firewall-drop

Le binaire firewall-drop ajoute une règle iptables pour bloquer l'IP attaquante en entrée et en transit. Après expiration du timeout (180 secondes), la règle est automatiquement supprimée.

Note : le blocage s'applique aux nouvelles connexions uniquement. Les connexions TCP déjà établies au moment du blocage ne sont pas interrompues.

7. Erreurs rencontrées

Erreur	Cause	Solution
Pas d'alerte dans les logs	/var/log/auth.log absent sur Debian 13	Configurer journald dans agent.conf
disable-account sans effet	Champ dstuser absent dans l'alerte 5405	Limitation connue de Wazuh
host-deny sans effet	TCP Wrappers deprecated sur Debian 13	Utiliser firewall-drop à la place

Erreur	Cause	Solution
Active response en boucle	Filtre level trop bas déclenche l'AR sur ses propres logs	Supprimer le filtre level de la config

8. Commandes utiles

bash

```
# Vérifier les règles iptables actives
sudo iptables -L INPUT -n

# Vider toutes les règles iptables
sudo iptables -F

# Surveiller les active responses en temps réel
sudo tail -f /var/ossec/logs/active-responses.log

# Surveiller les alertes en temps réel
sudo tail -f /var/ossec/logs/alerts/alerts.log | grep -E "5763|651"

# Vérifier la configuration active-response
sudo grep -A5 "active-response" /var/ossec/etc/ossec.conf
```

Wazuh — Intégration GLPI

1. Contexte et objectif

L'objectif est de créer automatiquement des tickets GLPI lorsque Wazuh détecte une alerte critique, sans intervention humaine. Le module Integrations de Wazuh permet d'exécuter un script externe à chaque alerte correspondant à des règles définies.

Architecture

Agent (ex: SRV-DOCKER)

↓ alerte critique détectée (brute force, FIM, CVE...)

Manager (srvwazuh)

↓ wazuh-integratord appelle le script

/var/ossec/integrations/custom-glpi

↓ appel API REST GLPI Legacy v1

GLPI → ticket créé automatiquement

2. Prérequis GLPI

2.1 Activation de l'API

Dans GLPI, aller dans Configuration → Générale → API :

- Activer l'API REST legacy : activé
- Activer la connexion avec un jeton externe : activé

2.2 Compte de service

Créer un utilisateur local GLPI dédié avec le login wazuh-api, le profil Technicien, et générer un user_token depuis Mon compte → Clés API.

2.3 Catégories ITIL

Créer les catégories dans Configuration → Intitulés → Assistance → Catégories ITIL :

Catégorie	Parent	ID
Sécurité	—	19
Intrusion	Sécurité	20
FIM - Intégrité fichiers	Sécurité	21
Vulnérabilité	Sécurité	23

2.4 Résolution DNS

Le serveur srvwazuh résolvait glpi.c1-04.lan vers l'IP du proxy (172.18.6.1) non joignable depuis le réseau 172.18.5.0/24. Correction par entrée statique dans /etc/hosts :

bash

```
sudo bash -c 'echo "172.18.5.174 glpi.c1-04.lan" >> /etc/hosts'
```

3. Script d'intégration

3.1 Emplacement et permissions

Le script doit être placé dans /var/ossec/integrations/ avec les permissions suivantes :

bash

```
sudo chmod 750 /var/ossec/integrations/custom-glpi
sudo chown root:wazuh /var/ossec/integrations/custom-glpi
```

Créer également le fichier de déduplication :

bash

```
sudo touch /var/ossec/logs/glpi_processed.log
sudo chown wazuh:wazuh /var/ossec/logs/glpi_processed.log
```

3.2 Script complet

python

```
#!/usr/bin/env python3
import sys
import json
import requests
import urllib3
urllib3.disable_warnings()

APP_TOKEN = "<app_token_glpi>"
USER_TOKEN = "<user_token_wazuh_api>"
GLPI_URL = "http://glpi.c1-04.lan/api.php/v1"
PROCESSED_FILE = "/var/ossec/logs/glpi_processed.log"

CATEGORY_SECURITY = 19
CATEGORY_INTRUSION = 20
CATEGORY_FIM = 21
CATEGORY_CVE = 23

CRITICAL_RULES = ["5763", "5720", "5405", "23501", "23502"]
CRITICAL_PATHS = [
    "/etc/passwd", "/etc/shadow", "/etc/sudoers",
    "/etc/ssh/", "/etc/cron", "/var/ossec/etc/",
    "/usr/bin/", "/usr/sbin/", "/bin/", "/sbin/"
]

def is_critical(alert):
    rule_id = alert.get("rule", {}).get("id", "")
    level = alert.get("rule", {}).get("level", 0)
    path = alert.get("syscheck", {}).get("path", "")

    if rule_id in ["23501", "23502"]:
        cvss = float(alert.get("data", {}).get("cvss3_score", 0))
```

```
        or alert.get("data", {}).get("cvss2_score", 0) or 0)
    return cvss >= 9.0
```

```
if rule_id in CRITICAL_RULES:
    return True
if level >= 10:
    return True
if path and any(p in path for p in CRITICAL_PATHS):
    return True
return False
```

```
def already_processed(alert_id):
    try:
        with open(PROCESSED_FILE, "r") as f:
            return alert_id in f.read()
    except:
        return False
```

```
def mark_processed(alert_id):
    with open(PROCESSED_FILE, "a") as f:
        f.write(alert_id + "\n")
```

```
def get_session_token():
    r = requests.get(f"{GLPI_URL}/initSession",
        headers={"App-Token": APP_TOKEN,
            "Authorization": f"user_token {USER_TOKEN}"},
        verify=False)
    return r.json()["session_token"]
```

```
def create_ticket(session_token, alert):
    rule_id = alert.get("rule", {}).get("id", "N/A")
    rule_desc = alert.get("rule", {}).get("description", "N/A")
    src_ip = alert.get("data", {}).get("srcip", "N/A")
    agent = alert.get("agent", {}).get("name", "N/A")
    level = alert.get("rule", {}).get("level", 0)
    path = alert.get("syscheck", {}).get("path", "")
    timestamp = alert.get("timestamp", "N/A")
    cve = alert.get("data", {}).get("cve", "")
    package = alert.get("data", {}).get("package", "")
    cvss = (alert.get("data", {}).get("cvss3_score", ""))
```

```
or alert.get("data", {}).get("cvss2_score", ""))
```

```
content = (f"Alerte Wazuh - Incident de securite\n"
```

```
    f"Timestamp : {timestamp}\n"
```

```
    f"Agent    : {agent}\n"
```

```
    f"Regle    : {rule_id} - {rule_desc}\n"
```

```
    f"Niveau   : {level}\n"
```

```
    f"IP source : {src_ip}")
```

```
if path:
```

```
    content += f"\nFichier : {path}"
```

```
if cve:
```

```
    content += f"\nCVE      : {cve}"
```

```
if package:
```

```
    content += f"\nPackage : {package}"
```

```
if cvss:
```

```
    content += f"\nCVSS    : {cvss}"
```

```
urgency = 4 if level >= 10 else 3
```

```
if path:
```

```
    category = CATEGORY_FIM
```

```
elif rule_id in ["5763", "5720"]:
```

```
    category = CATEGORY_INTRUSION
```

```
elif rule_id in ["23501", "23502"]:
```

```
    category = CATEGORY_CVE
```

```
else:
```

```
    category = CATEGORY_SECURITY
```

```
requests.post(f"{GLPI_URL}/Ticket",
```

```
    headers={
```

```
        "App-Token": APP_TOKEN,
```

```
        "Session-Token": session_token,
```

```
        "Content-Type": "application/json"
```

```
    },
```

```
    json={"input": {
```

```
        "name": f"[Wazuh] {rule_desc} - {agent}",
```

```
        "content": content,
```

```
        "urgency": urgency,
```

```
        "priority": urgency,
```

```
"status": 1,  
  "itilcategories_id": category  
}},  
verify=False)
```

try:

```
alert_file = open(sys.argv[1])  
alert = json.load(alert_file)  
alert_file.close()  
alert_id = alert.get("id", "")  
if is_critical(alert) and not already_processed(alert_id):  
    session = get_session_token()  
    create_ticket(session, alert)  
    mark_processed(alert_id)
```

except Exception as e:

```
with open("/var/ossec/logs/integrations.log", "a") as f:  
    f.write(f"custom-glpi error: {str(e)}\n")  
sys.exit(1)
```

4. Configuration ossec.conf

Ajout dans `/var/ossec/etc/ossec.conf` :

xml

```
<integration>  
  <name>custom-glpi</name>  
  <rule_id>5763,5720,5405,550,551,554,23501,23502</rule_id>  
  <alert_format>json</alert_format>  
</integration>
```

Redémarrer le manager pour appliquer :

bash

```
sudo systemctl restart wazuh-manager
```

5. Logique de tri des alertes

Condition	Catégorie GLPI
Alerte FIM sur chemin critique	FIM - Intégrité fichiers
Règles 5763 ou 5720 (SSH)	Intrusion
Règles 23501 ou 23502 avec CVSS supérieur ou égal à 9.0	Vulnérabilité
Autres alertes critiques	Sécurité

L'urgence du ticket est définie à Haute si le niveau Wazuh est supérieur ou égal à 10, et Moyenne dans les autres cas.

6. Erreurs rencontrées

Erreur	Cause	Solution
Timeout connexion GLPI	DNS résolvait vers le proxy 172.18.6.1 non joignable	Ajout entrée statique dans /etc/hosts
ERROR_INVALID_PARAMETER	API v2.2 utilise OAuth/JWT incompatible avec le user_token classique	Utiliser l'API Legacy v1
404 Not Found	Mauvais chemin URL	Chemin correct : /api.php/v1/
SSL WRONG_VERSION_NUMBER	GLPI migré en LXC sans HTTPS	Changer l'URL en http://
Tickets en double	Wazuh appelle le script plusieurs fois pour la même alerte	Déduplication par ID d'alerte
Script sans effet	ID d'alerte vide dans le fichier de test	Ajouter le champ id dans le JSON de test

7. Tests

Test manuel brute force

```
bash
```

```
cat > /tmp/test_alert.json << 'EOF'
{
  "id": "test_001",
  "timestamp": "2026-05-11T13:00:00",
  "rule": {"id": "5763", "description": "sshd: brute force", "level": 10},
  "agent": {"name": "Agent-Docker", "ip": "172.18.5.172"},
  "data": {"srcip": "172.18.6.1"}
}
EOF

sudo python3 /var/ossec/integrations/custom-glpi /tmp/test_alert.json
```

Test CVE critique

bash

```
cat > /tmp/test_cve.json << 'EOF'
{
  "id": "test_cve_001",
  "timestamp": "2026-05-11T14:00:00",
  "rule": {"id": "23501", "description": "CVE-2024-1234 detected", "level": 10},
  "agent": {"name": "Agent-Docker", "ip": "172.18.5.172"},
  "data": {"cve": "CVE-2024-1234", "package": "openssl", "cvss3_score": "9.8"}
}
EOF

sudo python3 /var/ossec/integrations/custom-glpi /tmp/test_cve.json
```

Vérification

bash

```
# Erreurs du script
sudo cat /var/ossec/logs/integrations.log

# Statut de l'intégration
sudo cat /var/ossec/logs/ossec.log | grep "custom-glpi" | tail -10
```

Alertes déjà traitées

```
sudo cat /var/ossec/logs/gltpi_processed.log
```